

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Mendoza

Ingeniería en Sistemas de Información

Guía Práctica N.º 6

Interbloqueos (Deadlock)

Cátedra: **Sistemas Operativos** • Unidad Temática N.º 6

Trabajo grupal — 4 a 5 integrantes

Docentes: Ing. Martínez • Lic. Faccio • Ing. Marques • Ing. Ortiz • Ing. Pennisi • Ing. Edera

Ciclo lectivo 2026

Consigna general

Resolver los ejercicios sincrónicos y asincrónicos sobre interbloqueos: grafos de asignación de recursos, algoritmo del banquero y los métodos de prevención, evitación y detección. Consultar la bibliografía recomendada en el cronograma de actividades prácticas.

Carácter de la actividad: Grupal

Parte A: Ejercicios Sincrónicos

1. Analice el gráfico de asignación de recursos de la **Fig. 1** y responda las siguientes preguntas:

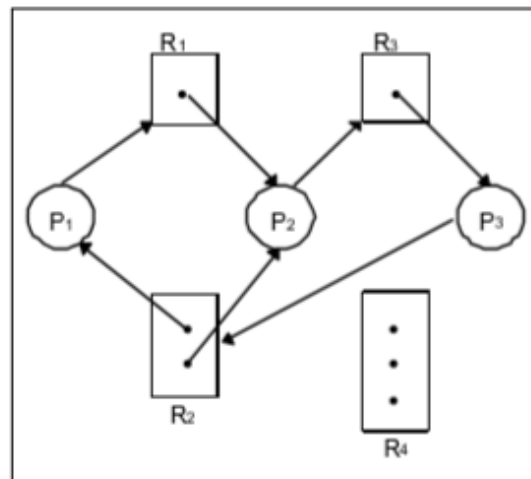


Figura 1: Fig. 1 — Grafo de asignación de recursos

- ¿Cuántos recursos hay y cuántos ejemplares de cada uno?
- ¿Cuál es el estado de los procesos?
- ¿Hay interbloqueo?

2. Indique cuál es el número máximo de procesos que se pueden ejecutar en un sistema que cuenta con recursos de dos tipos diferentes, con 3 ejemplares disponibles para cada recurso. Considere que cada proceso necesitará, durante su ejecución, de una unidad de cada tipo de recursos.

3. Suponiendo que se tiene un estado de ejecución inicial de procesos y asignación de recursos como el indicado en la **Tabla 1**. Considerando que se dispone de un solo recurso con un total de 12 instancias en el sistema.

Tabla 1 — Estado de asignación:

Proceso	Asignados	Máx. Necesarios
A	2	8
B	3	7
C	1	6
D	3	5

- Elabore una secuencia de ejecución de procesos que permita finalizar la ejecución de todos los procesos.
- Elabore una secuencia de ejecución de los procesos que lleve el sistema a un estado inseguro.

Comente y grafique adecuadamente las soluciones propuestas.

Nota (de la guía): Para elaborar la solución recurra al punto 6.5.2 del libro de Tanenbaum.

4. Considere el algoritmo del banquero de 1 solo recurso, con 4 procesos con los siguientes créditos: **A 3; B 6; C 2; D 5**. El sistema dispone de un **total de 16 instancias** del recurso.

Considerando que el banco **reserva el 50 % de las instancias totales (redondeando hacia abajo) y nunca las presta** —es decir, el **pool prestable** es la mitad del total, y el **Disponible** contra el que se analiza la seguridad es lo que queda sin prestar de ese pool— y si las peticiones y devoluciones se realizan del siguiente modo:

A B A -A C D D B -C B

Nota: -X implica devolución del recurso.

- Elabore la tabla de asignación de recursos que responda al estado en el momento indicado en el párrafo precedente.
- Indique si el estado es seguro o inseguro. Justifique su respuesta.
- Agregue una o más peticiones para llevar el estado a situación de inseguro y explique por qué se llega a esta situación.

Parte B: Ejercicios Asincrónicos

- Defina qué se entiende por interbloqueo y dé al menos 2 ejemplos en los que haya múltiples procesos y recursos.
- Explique las diferencias entre interbloqueo, bloqueo activo e inanición.
- Según la bibliografía principal, los inconvenientes del **método de prevención de la condición de retención y espera** (hold and wait) son: a) Pobre uso de los recursos; b) debido a la metodología empleada al solicitar recursos puede que ocurra espera indefinida para obtenerlos.
 - Justifique a)
 - Describa la metodología empleada para solicitar recursos.
- Explique, en forma detallada, el **método de prevención que ataca la condición de no expropiación**.
- Espera circular.
 - Mencione y explique cuáles son las condiciones que se deben cumplir para que no se produzca “Espera circular”.
 - ¿Cuál es el principal inconveniente de este método?
- Ejemplifique una situación de los sistemas operativos donde se produzca inanición, pero no haya interbloqueo, y una situación donde se produzca inanición e interbloqueo.
- Considerando el siguiente algoritmo:

```
semaforo recurso_1;
semaforo recurso_2;

void proceso_A(void) {
    down(&recurso_1);
    down(&recurso_2);
    usar_ambos_recurso();
    up(&recurso_2);
    up(&recurso_1);
}
```

```
}
```

```
void proceso_B(void) {  
    down(&recurso_2);  
    down(&recurso_1);  
    usar_ambos_recursos();  
    up(&recurso_1);  
    up(&recurso_2);  
}
```

- a) Describa una secuencia de ejecución en la que no se produzca interbloqueo.
- b) Describa una secuencia de ejecución en la que sí se produzca interbloqueo.